The late delivery the dispatcher never saw

Tokens burned. Time lost. Nothing logged.

Sonakshi Pandey

TIFFIN

Food-delivery operations · 6 AI agents · 11 MCP tool servers · 1 dispatcher question in 12 comes back blind

THE PROBLEM

A dispatcher asks Tiffin's delivery-routing agent which of a courier's deliveries are running late. They are asking so they can act while there is still time: reroute the courier, warn the customer, comp the order.

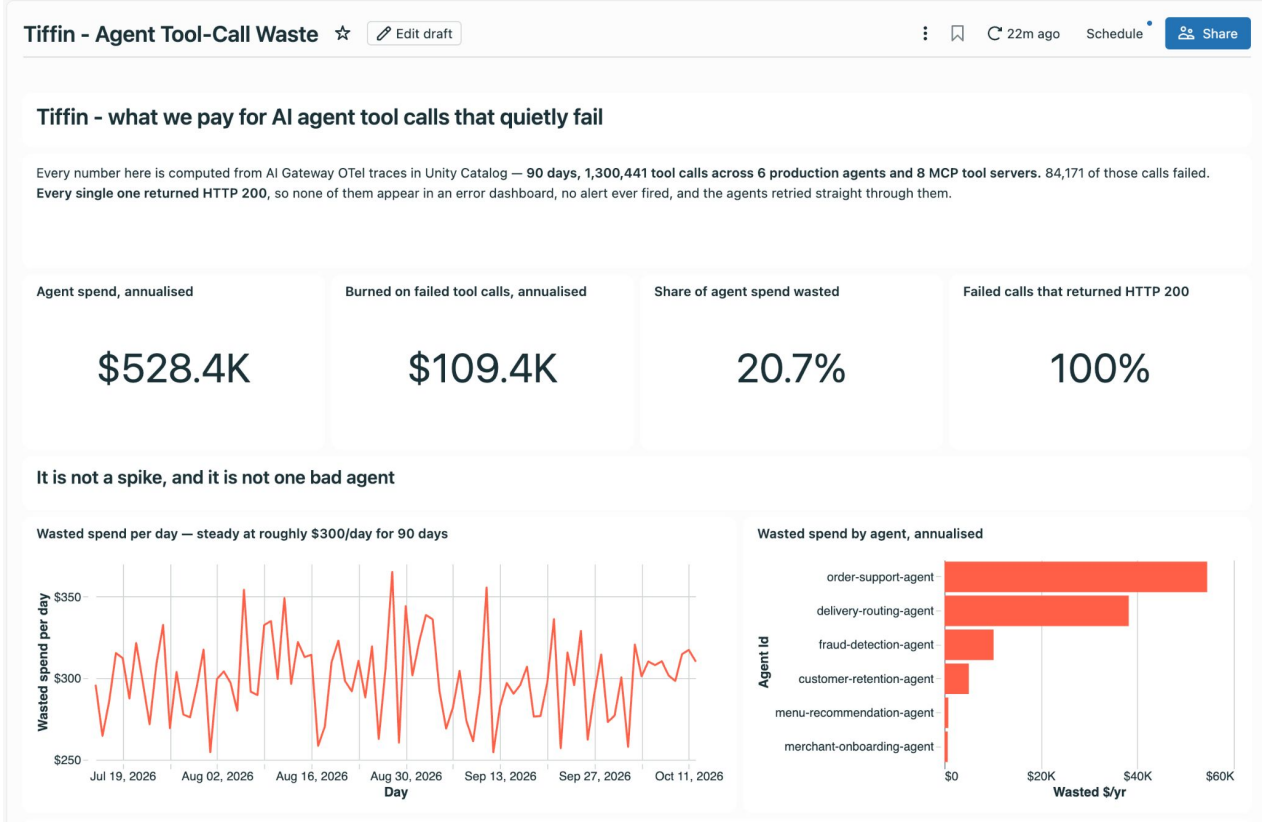
One question in twelve runs into a broken MCP tool contract. The agent retries, turns double, and the answer takes 20.6 seconds instead of 8.6 — or never arrives. The courier stays late and nobody is told.

Nothing alerts, because the tool call that failed returned HTTP 200.

THE SOLUTION

A guard model scores every tool call before dispatch and returns a verdict: coerce or allow. On coerce the agent rewrites the bad argument from the server's own declared schema, so the failure and its retry loop are prevented at dispatch rather than diagnosed afterwards.

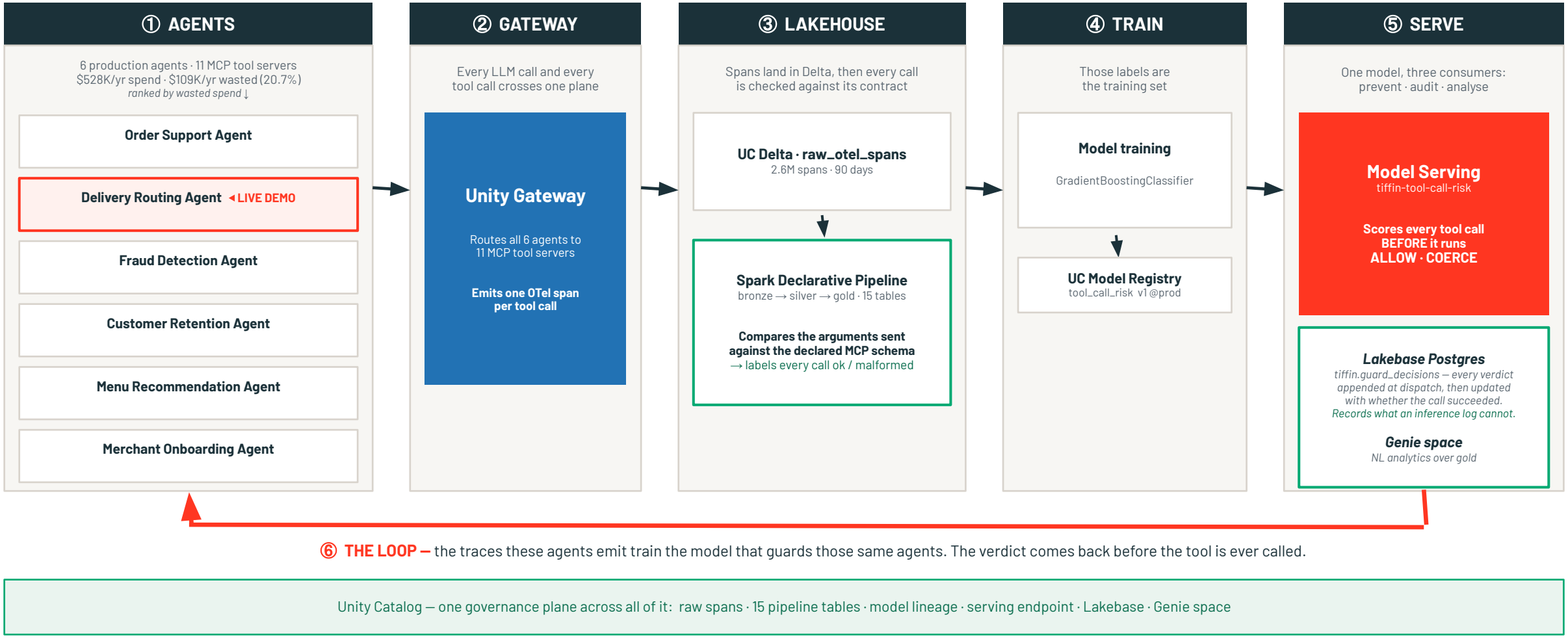
Head of Delivery Operations: dispatchers get the answer inside the window they can still act in. Platform team: 20.7% of agent spend recovered, $109K/yr. Tool-server owners: which contract to repair first, ranked by dollars, because cost per failed call varies 102x.



©2026 Databricks, Inc. — All rights reserved

Six agents. One Gateway. One closed loop.

All six production agents route through the AI Gateway → it emits an OTel span for every tool call → Lakeflow compares the arguments sent against the declared MCP schema → those labels train the model → the model scores the next call before it runs.



⑥ **THE LOOP —** the traces these agents emit train the model that guards those same agents. The verdict comes back before the tool is ever called.

Unity Catalog — one governance plane across all of it: raw spans · 15 pipeline tables · model lineage · serving endpoint · Lakebase · Genie space

©2026 Databricks, Inc. — All rights reserved

What was observed

A dispatcher asks which of a courier's deliveries are running late in order to act on it: reroute, warn the customer, comp the order. 8.0% of those questions hit a tool call that failed while returning HTTP 200. Here is what that costs the dispatch floor.

Measured, per dispatcher session	Clean session	Hit a silent tool failure	What it means on the dispatch floor
Dispatcher sessions	223,711	**8.0% · 19,401**	One question in twelve comes back blind. Not an edge case.
Median time to an answer	8.6 s	**20.6 s**	2.4x the wait, inside the window where a reroute still changes the outcome.
Agent turns to reach it	3.82	**7.47**	The retry loop, not slower tools: per-turn latency is drawn from the same distribution either way.
Cost per question (wasted)	$0.46 ($0.02)	**$1.24 ($1.14)**	2.7x the cost, and 92% of a broken session's spend buys nothing.

And sometimes the answer never arrives at all

Recorded A/B run, same question, guard off then guard on. GUARD OFF: the agent's page-2 tool call was rejected by the server, it did not retry, and it reported no late deliveries. D-4410315 was 18 minutes late and the dispatcher was never told. GUARD ON: the malformed argument was coerced from the server's own declared schema, the call succeeded first time, and the late delivery surfaced.

Measured across 2.6M Gateway OTel spans · 243,112 sessions · 90 days · 6 production agents. Every figure above is a committed query result in evidence/09_delivery_operations_impact.md and evidence/05_agent_ab_run.md.
Guard accuracy, scored leave-one-bug-family-out so that no bug family in a test fold appears in training: AUC 0.825. A hand-written rule set scored the same way: 0.604. An error-log baseline: 0.000, because the failures return HTTP 200.
Not claimed: the conversion from a missed reroute window into on-time-delivery percentage. That coefficient is the operator's own, and the four rows above move regardless of what it turns out to be.

©2026 Databricks, Inc. — All rights reserved

How this was built

Roughly 20 hours over six days on a Databricks dogfood workspace. 14 commits, sole author. Claude Code (Opus) wrote most of the code, driving the Databricks CLI and REST APIs directly rather than handing me scripts to run. The framing, the modelling calls and every decision below are mine.

① THE WORKFLOW — none of it was built in parallel, because each stage exposed a fault in the one before it

Generate the trace corpus → **Lakeflow pipeline** → **Train + evaluate** → **Serve the model** → **Wrap an agent around it** → **Add the audit trail**

The pipeline revealed that the generator's retry signature was wrong. Training revealed that the pipeline's labels were separable for the wrong reason. The agent revealed that the model's three-way verdict only had two behaviours behind it.
The loop that actually ran, dozens of times a day: make a change, run it as a real job, read the output, write the output into evidence/. Anything not verified by output was treated as not done.

② WHERE AI CHANGED HOW THE WORK GOT DONE

Throwaway verification became free.
Two-minute probes you would normally just assume. Three of my assumptions turned out to be wrong.

Claims got re-queried, not remembered.
Every number recomputed rather than recalled. That habit is why evidence/ exists at all.

Attacking my own results became routine.
An early model scored near 1.0 on a random split; treating that as a red flag rather than a win produced the leave-one-bug-family-out protocol.

The deck became code.
Built through the Slides API, so a layout change is a diff instead of an afternoon of dragging boxes.

③ WHERE I HAD TO OVERRULE IT

It built to the checklist.
The first Lakebase layer was a nine-row copy of a Delta table. I cut it. The rebuild was scoped against what inference logs already capture.

It trusted its own synthetic data.
It offered a failure-to-escalation correlation as a finding. That correlation was hand-written into the generator, so reading it back was circular. Kept off this deck.

It measured the wrong thing first.
Summed tool latency made broken sessions look faster, because validator rejections return quickly. The honest measure is turn + tool latency, at the median.

It needed to be told to stop.
A fix-queue panel, a second app tab, a third verdict with no behaviour behind it. All built, all cut.

WHAT IS REAL — every platform component is live on e2-dogfood: AI Gateway OTel span schema · Lakeflow Spark Declarative Pipeline (15 tables) · Unity Catalog · MLflow + UC Model Registry · Model Serving · Lakebase verdict log · Genie space · Databricks App · AI/BI dashboard.
WHAT IS SYNTHETIC — the Tiffin corpus: 2.6M spans generated from a seeded bug registry against the real Gateway span schema. Real shape, fabricated volume, because real Gateway traces are not mine to publish.
The full account, including the trade-offs and what the mistakes above cost, is in BUILD.md and evidence/10_build_process.md.

©2026 Databricks, Inc. — All rights reserved

Where this goes

The guard reads exactly two things: a Gateway OTel span and the tool server's own declared MCP schema. Nothing in it is Tiffin-specific and nothing is per-agent, which is why it scales sideways with no retraining — enrolment is routing through the Gateway, not integrating with us.

TODAY · one agent guarded

delivery-routing-agent scores every tool call before dispatch. Every verdict lands in the Lakebase verdict log together with whether the call then actually succeeded. Live on e2-dogfood today.

→

NEXT · all six agents

Same endpoint, same model, no retraining. All six agents already route through the Gateway and all 11 tool servers already publish schemas. Brings the full 20.7% of $528K/yr into scope.

→

THEN · anything on the Gateway

Any agent built after this inherits the guard on the day it ships. The platform team stops discovering silent tool failures from a cost report.

④ THE CLOSING LOOP — the guard is not the end state, it is the instrument that finds the bug

Verdict log ranks failing tool contracts by wasted dollars → **Tool-server owner repairs the contract at source** → **That bug family stops occurring at all** → **The guard's coerce volume for it falls to zero** → **Wasted spend share drops below 20.7%, ranking moves on**

Why the ranking is the point: cost per failed call varies 102x across tool servers, from $2.35 on payment-gateway.create_refund down to $0.023 on merchant-catalog-api.update_merchant. Fixing the most frequent broken contract is not the same as fixing the most expensive one, and the verdict log is what tells them apart.
The number that says it is working: wasted spend falling below 20.7%. If the guard is doing its job, its own coerce volume should shrink over time rather than grow.

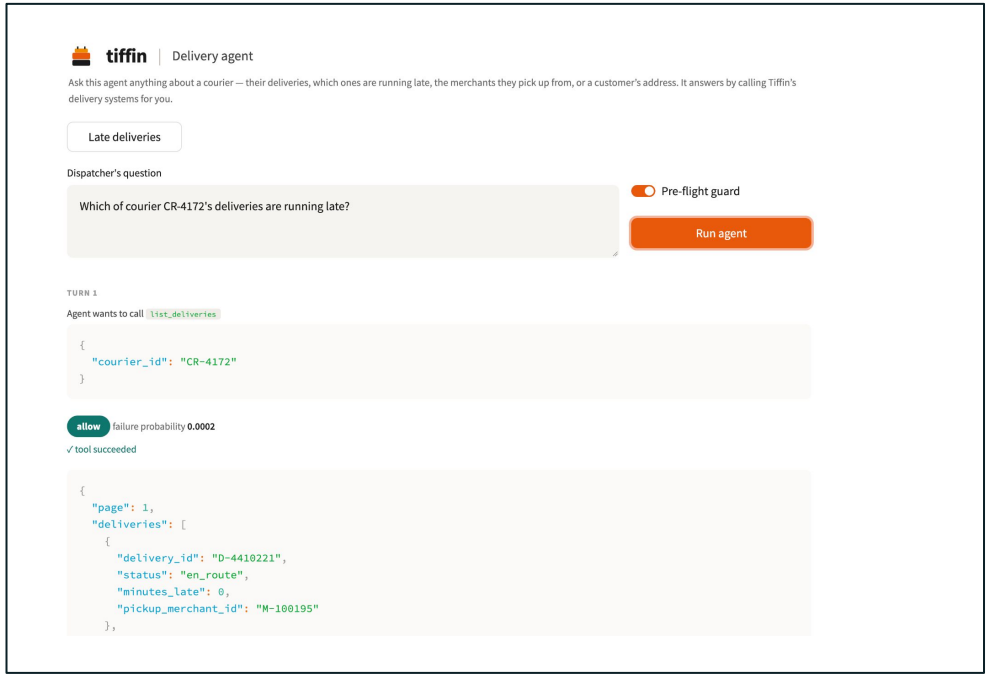
THE HONEST LIMIT
The weakest leave-one-bug-family-out fold scores AUC 0.336, below random. The mean of 0.825 is carried by the bug families the model generalises to; it does not generalise to all of them.
So the guard is a first line of defence and not a substitute for repairing tool contracts. That is precisely why the loop above is the plan, and why shipping the guard on its own would not be.

©2026 Databricks, Inc. — All rights reserved

Links



Genie Space



Delivery Agent

REAL — functional today on e2-dogfood
Gateway OTel schema · Lakeflow pipeline (15 tables) · MLflow model + leave-one-bug-family-out eval · UC registry · Model Serving endpoint · Lakebase verdict log · Genie space · Databricks App · AI/BI dashboard
github.com/29sonakshi/stop-agent-waste — code + committed execution evidence

MOCKED — only to make the story demoable
Synthetic Tiffin trace corpus (2.6M spans generated from a seeded bug registry) · 5 in-process MCP tool mocks inside the demo app

©2026 Databricks, Inc. — All rights reserved